Formal Verification of CategoricalCellularAutomata.jl

Simon Frost

2026-11-04

Contents

Formal verification of CategoricalCellularAutomata.jl 1

Configurations, shifts and pullbacks 2

Cellular automata over a group 3

The cartesian category of cellular automata over a group 4

Curtis–Hedlund–Lyndon 7

Linear cellular automata and the group algebra 8

Generalised cellular automata and pullbacks 9

Gluing spaces 10

Moving models between spaces 12

Formal verification of CategoricalCellularAutomata.jl

This library proves, in Lean 4 with Mathlib, the laws that `CategoricalCellularAutomata.jl` relies on and that its test suite checks numerically. The definitions are the mathematical objects the Julia types implement, over any group (finite or not), with any alphabet; the Julia package works with finite groups and alphabets, so its computations are instances.

Lean	Julia	statement
`Config G A`	`Configuration`	a configuration `G → A`
`shift g x`	`shift(x, g)`	`(g · x)(h) = x(h g)`, a left action (`shift_one`, `shift_shift`)
`pullback φ x`	`pullback(φ, X)`	`x ∘ φ`, contravariantly functorial (`pullback_id`, `pullback_comp`)
`ca S μ`	`CellularAutomaton(G, S, μ)`	`τ(x)(g) = μ((x(s g))_{s ∈ S})`
`ca_shift`	`is_equivariant`	every cellular automaton commutes with the shifts
`ca_eq_iff`	`σ == τ`	two automata are equal when they agree at the identity
`ca_id`, `ca_comp`	`id`, `compose`	the identity, and composites with memory set `T S`
`ca_pair`	`pair`	pairing over the disjoint union of memory sets
`CAObj G`, `IsCA`	`FullShift`, `CellularAutomaton`	alphabets, and the maps of configurations that are cellular automata

Lean	Julia	statement
`CartesianMonoidalCategory (CAObj G)`	the Catlab `@instance` of `ThCartesianCategory`	CA(G) is a cartesian monoidal category: `compose`, `id`, `otimes`, `munit`, `proj1`, `proj2`, `pair`
`isCA_iff_shift_comm_and_local`	`is_equivariant`, `cellular_automaton`	Curtis–Hedlund–Lyndon: a cellular automaton is an equivariant map that is local at the identity
`isCA_iff_shift_comm`	`cellular_automaton`, `minimal_neighbourhood`	over a finite group, every equivariant map is a cellular automaton
`linCA a`	`LinearCA(G, coefficients; p)`	`τ(x)(g) = Σ_s a_s x(s g)`
`linCA_mul`, `linCA_pow`	`compose`, `^` for `LinearCA`	composition and powers are products in the group algebra
`gca φ S μ`	`GeneralizedCA(φ, S, μ)`	`T(x)(h) = μ((x(s φ(h)))_{s ∈ S})`
`gca_eq_pullback_ca`	`factorize`	`T = φ∗ ∘ τ`
`pullback_ca`	`pullback`, `SpaceMorphism`	pullback commutes with local rules: `φ∗ ∘ τ_{φ(S)} = τ_S ∘ φ∗`
`gca_comp`	`compose`	generalised automata compose along `φ ∘ ψ`
`gluedConfigEquiv`, `exists_unique_glued_config`	`glue`	configurations of a glued space are the compatible families on its patches (a limit)
`step_restrict`, `step_eq_of_cover`	`glue`, `apply_local`	the landscape's step on a patch's interior is the patch's step
`SpaceMap`, `SpaceMap.run_pullback`	`SpaceMorphism`, `pullback(f, prob; space)`	a problem moved along a map of spaces (slots reordered, the image's rule and random numbers) runs the pullback of the original run

Every theorem in the default target is free of `sorry`; `Audit.lean` prints the axioms each main theorem uses (only `propext`, `Classical.choice` and `Quot.sound` are expected).

Configurations, shifts and pullbacks

```
import Mathlib.Algebra.Group.Hom.Defs
```

A configuration over a group `G` with values in an alphabet `A` is a function `x : G → A` (`Configuration` in `CategoricalCellularAutomata.jl`). The group acts on configurations on the left by `(g · x)(h) = x (h * g)` (`shift(x, g)`), and a homomorphism `φ : H →* G` pulls a configuration over `G` back to one over `H` by precomposition (`pullback(φ, X)`).

```
namespace CategoricalCellularAutomataProofs

/-- A configuration over `G` with values in `A`: the state of a cellular automaton. -/
abbrev Config (G : Type*) (A : Type*) := G → A

variable {G H K A : Type*}

section Shift

variable [Monoid G]

/-- The shift `g · x`, with `(g · x)(h) = x (h * g)` (`shift(x, g)`). -/
def shift (g : G) (x : Config G A) : Config G A := fun h ⇒ x (h * g)

@[simp] theorem shift_apply (g h : G) (x : Config G A) : shift g x h = x (h * g) := rfl

/-- The identity acts trivially. -/
```

```
@[simp] theorem shift_one (x : Config G A) : shift 1 x = x := by
  funext h; simp [shift]

/-- Shifting is a left action: `g · (g' · x) = (g g') · x`. -/
theorem shift_shift (g g' : G) (x : Config G A) : shift g (shift g' x) = shift (g * g') x := by
  funext h; simp [shift, mul_assoc]

end Shift

section Pullback

/-- The pullback `φ* x = x ∘ φ` of a configuration along a map of groups (`pullback(φ, X)`). -/
def pullback [MulOneClass H] [MulOneClass G] (φ : H →* G) (x : Config G A) : Config H A :=
  fun h ⇒ x (φ h)

@[simp] theorem pullback_apply [MulOneClass H] [MulOneClass G] (φ : H →* G) (x : Config G A)
    (h : H) : pullback φ x h = x (φ h) := rfl

/-- Pulling back along the identity changes nothing. -/
@[simp] theorem pullback_id [MulOneClass G] (x : Config G A) :
    pullback (MonoidHom.id G) x = x := rfl

/-- Pullback is contravariantly functorial: `(φ ∘ ψ)* = ψ* ∘ φ*`. -/
theorem pullback_comp [MulOneClass K] [MulOneClass H] [MulOneClass G] (φ : H →* G)
    (ψ : K →* H) (x : Config G A) :
    pullback (φ.comp ψ) x = pullback ψ (pullback φ x) := rfl

/-- Pullback intertwines the shifts: `φ* (φ(h) · x) = h · φ* x`. -/
theorem pullback_shift [Monoid H] [Monoid G] (φ : H →* G) (h : H) (x : Config G A) :
    pullback φ (shift (φ h) x) = shift h (pullback φ x) := by
  funext k; simp [pullback, shift, map_mul]

end Pullback

end CategoricalCellularAutomataProofs
```

Cellular automata over a group

```
import CategoricalCellularAutomataProofs.Configurations
```

A cellular automaton over `G` with memory set `S` and local rule `μ` maps a configuration `x` to `τ(x)(g) = μ((x(s g))_{s ∈ S})` (`CellularAutomaton(G, S, μ)`). The memory set is an ordered list of group elements, here a family `S : I → G` indexed by any type `I` (in Julia, `I = Fin (length S)`), and the local rule reads the values at `S i * g` in that order.

This file proves that every cellular automaton commutes with the shifts, that a cellular automaton is determined by its values at the identity (which is how `σ == τ` decides equality), that the identity map is a cellular automaton, that the composite of two is a cellular automaton over the product of their memory sets (`compose`), and that pairing two cellular automata is one cellular automaton over the disjoint union of their memory sets (`pair`).

```
namespace CategoricalCellularAutomataProofs

variable {G A B C : Type*} {I J : Type*}

/-- The cellular automaton with memory set `S` and local rule `μ`:
`τ(x)(g) = μ (fun i ⇒ x (S i * g))`. -/
def ca [Mul G] (S : I → G) (μ : (I → A) → B) (x : Config G A) : Config G B :=
  fun g ⇒ μ (fun i ⇒ x (S i * g))

@[simp] theorem ca_apply [Mul G] (S : I → G) (μ : (I → A) → B) (x : Config G A) (g : G) :
    ca S μ x g = μ (fun i ⇒ x (S i * g)) := rfl

section Monoid
```

```
variable [Monoid G]

/-- **Equivariance.** A cellular automaton commutes with every shift: `τ (g · x) = g · τ x`. -/
theorem ca_shift (S : I → G) (μ : (I → A) → B) (g : G) (x : Config G A) :
    ca S μ (shift g x) = shift g (ca S μ x) := by
  funext h; simp [ca, shift, mul_assoc]

/-- A shift-equivariant map is determined by its values at the identity:
`τ(x)(g) = τ(g · x)(1)`. -/
theorem apply_eq_apply_one_shift (τ : Config G A → Config G B)
    (hτ : ∀ g x, τ (shift g x) = shift g (τ x)) (x : Config G A) (g : G) :
    τ x g = τ (shift g x) 1 := by
  rw [hτ]; simp

/-- **Equality at the identity.** Two shift-equivariant maps agree when they agree at the
identity element on every configuration; this is how `σ == τ` compares cellular automata. -/
theorem eq_of_apply_one (σ τ : Config G A → Config G B)
    (hσ : ∀ g x, σ (shift g x) = shift g (σ x)) (hτ : ∀ g x, τ (shift g x) = shift g (τ x))
    (h : ∀ x, σ x 1 = τ x 1) : σ = τ := by
  funext x g
  rw [apply_eq_apply_one_shift σ hσ, apply_eq_apply_one_shift τ hτ, h]

/-- Two cellular automata are equal as maps exactly when they agree at the identity. -/
theorem ca_eq_iff {S : I → G} {T : J → G} {μ : (I → A) → B} {ν : (J → A) → B} :
    ca S μ = ca T ν ↔ ∀ x, ca S μ x 1 = ca T ν x 1 := by
  constructor
  · intro h x; rw [h]
  · intro h
    exact eq_of_apply_one _ _ (fun g x ⇒ ca_shift S μ g x) (fun g x ⇒ ca_shift T ν g x) h

/-- The identity map is the cellular automaton with memory set `{1}` that reads its one
value. -/
theorem ca_id (x : Config G A) : ca (fun _ : Unit ⇒ (1 : G)) (fun p ⇒ p ()) x = x := by
  funext g; simp [ca]

/-- **Composition.** Running `τ = ca T ν` and then `σ = ca S μ` is the cellular automaton
whose memory set is the family of products `T j * S i` (the set `T S` of `compose`) and whose
local rule applies `ν` to each block and `μ` to the results. -/
theorem ca_comp (S : I → G) (μ : (I → B) → C) (T : J → G) (ν : (J → A) → B)
    (x : Config G A) :
    ca S μ (ca T ν x) =
      ca (fun p : I × J ⇒ T p.2 * S p.1) (fun q ⇒ μ (fun i ⇒ ν (fun j ⇒ q (i, j)))) x := by
  funext g; simp [ca, mul_assoc]

/-- The composite is again equivariant, as a cellular automaton must be. -/
theorem ca_comp_shift (S : I → G) (μ : (I → B) → C) (T : J → G) (ν : (J → A) → B) (g : G)
    (x : Config G A) :
    ca S μ (ca T ν (shift g x)) = shift g (ca S μ (ca T ν x)) := by
  rw [ca_shift, ca_shift]

/-- **Pairing.** The pair of two cellular automata over the same group is one cellular
automaton whose memory set is the disjoint union of theirs (`pair`, the union `S ∪ T`). -/
theorem ca_pair (S : I → G) (μ : (I → A) → B) (T : J → G) (ν : (J → A) → C)
    (x : Config G A) (g : G) :
    ca (Sum.elim S T) (fun p ⇒ (μ (fun i ⇒ p (Sum.inl i)), ν (fun j ⇒ p (Sum.inr j)))) x g =
      (ca S μ x g, ca T ν x g) := rfl

end Monoid

end CategoricalCellularAutomataProofs
```

The cartesian category of cellular automata over a group

```
import CategoricalCellularAutomataProofs.CellularAutomata
import Mathlib.CategoryTheory.Monoidal.Cartesian.Basic
import Mathlib.Data.Finite.Prod
import Mathlib.Data.Finite.Sum
```

The package makes the cellular automata over a group `G` a Catlab instance of `ThCartesianCategory`: the objects are full shifts `A^G` (one for each alphabet `A`), the morphisms are cellular automata, composition is `compose`, the product of `A^G` and `B^G` is `(A × B)^G` (`otimes`), with `proj1`, `proj2` and `pair`, and the unit `munit()` is the one-letter alphabet.

Here the category is built in Mathlib: an object is an alphabet, a morphism is a map of configurations that *is* a cellular automaton (has a finite memory set and a local rule), and Mathlib's `CartesianMonoidalCategory.ofChosenFiniteProducts` turns the terminal object and the binary products into the cartesian monoidal structure, with all its coherence laws (the associator, unitors and braiding are then the ones the package's strict products realise).

```
namespace CategoricalCellularAutomataProofs

open CategoryTheory Limits

universe u

variable (G : Type u) [Monoid G]

/-- A map of configurations is a cellular automaton when it has a finite memory set `S` and
a local rule `μ`. -/
def IsCA {A B : Type u} (f : Config G A → Config G B) : Prop :=
  ∃ (I : Type u) (_ : Finite I) (S : I → G) (μ : (I → A) → B), f = ca S μ

variable {G}

theorem isCA_ca {A B : Type u} {I : Type u} [Finite I] (S : I → G) (μ : (I → A) → B) :
    IsCA G (ca S μ) :=
  ⟨I, inferInstance, S, μ, rfl⟩

theorem isCA_id (A : Type u) : IsCA G (id : Config G A → Config G A) :=
  ⟨PUnit, inferInstance, fun _ ⇒ 1, fun p ⇒ p PUnit.unit, by funext x g; simp [ca]⟩

theorem isCA_comp {A B C : Type u} {f : Config G A → Config G B} {g : Config G B → Config G C}
    (hf : IsCA G f) (hg : IsCA G g) : IsCA G (g ∘ f) := by
  obtain ⟨J, _, T, ν, rfl⟩ := hf
  obtain ⟨I, _, S, μ, rfl⟩ := hg
  refine ⟨I × J, inferInstance, fun p ⇒ T p.2 * S p.1,
    fun q ⇒ μ (fun i ⇒ ν (fun j ⇒ q (i, j))), ?_⟩
  funext x
  exact ca_comp S μ T ν x

/-- A cellular automaton is equivariant: it commutes with every shift. -/
theorem IsCA.shift_comm {A B : Type u} {f : Config G A → Config G B} (hf : IsCA G f) (g : G)
    (x : Config G A) : f (shift g x) = shift g (f x) := by
  obtain ⟨I, _, S, μ, rfl⟩ := hf
  exact ca_shift S μ g x

/-- An alphabet, as an object of the category of cellular automata over `G` (the full shift
`A^G`). The group is a parameter of the type, so each group has its own category. -/
@[nolint unusedArguments]
structure CAObj (G : Type u) [Monoid G] where
  /-- the alphabet -/
  α : Type u

/-- The category of cellular automata over `G`: morphisms are the maps of configurations that
are cellular automata, composed as maps. -/
instance : Category (CAObj G) where
```

```
  Hom X Y := { f : Config G X.α → Config G Y.α // IsCA G f }
  id X := ⟨id, isCA_id X.α⟩
  comp f g := ⟨g.1 ∘ f.1, isCA_comp f.2 g.2⟩

@[ext] theorem CAObj.hom_ext {X Y : CAObj G} {f g : X → Y} (h : f.1 = g.1) : f = g :=
  Subtype.ext h

@[simp] theorem CAObj.id_val (X : CAObj G) : (𝟙 X : X → X).1 = id := rfl

@[simp] theorem CAObj.comp_val {X Y Z : CAObj G} (f : X → Y) (g : Y → Z) :
    (f ≫ g).1 = g.1 ∘ f.1 := rfl

/-- The one-letter alphabet, the unit `munit()`. -/
def unitObj : CAObj G := ⟨PUnit⟩

/-- The product alphabet `A × B` (`otimes`). -/
def prodObj (X Y : CAObj G) : CAObj G := ⟨X.α × Y.α⟩

/-- The first projection `proj1`: memory set `{1}`, rule `fst`. -/
def fstHom (X Y : CAObj G) : prodObj X Y → X :=
  ⟨ca (fun _ : PUnit ⇒ 1) (fun p ⇒ (p PUnit.unit).1), isCA_ca _ _⟩

/-- The second projection `proj2`: memory set `{1}`, rule `snd`. -/
def sndHom (X Y : CAObj G) : prodObj X Y → Y :=
  ⟨ca (fun _ : PUnit ⇒ 1) (fun p ⇒ (p PUnit.unit).2), isCA_ca _ _⟩

@[simp] theorem fstHom_apply (X Y : CAObj G) (x : Config G (X.α × Y.α)) (g : G) :
    (fstHom X Y).1 x g = (x g).1 := by
  simp [fstHom, ca]

@[simp] theorem sndHom_apply (X Y : CAObj G) (x : Config G (X.α × Y.α)) (g : G) :
    (sndHom X Y).1 x g = (x g).2 := by
  simp [sndHom, ca]

/-- The pairing `pair(f, g)` of two cellular automata is a cellular automaton (over the
disjoint union of their memory sets). -/
def pairHom {W X Y : CAObj G} (f : W → X) (g : W → Y) : W → prodObj X Y :=
  ⟨fun x h ⇒ (f.1 x h, g.1 x h), by
    obtain ⟨I, _, S, μ, hS⟩ := f.2
    obtain ⟨J, _, T, ν, hT⟩ := g.2
    refine ⟨I ⊕ J, inferInstance, Sum.elim S T,
      fun p ⇒ (μ (fun i ⇒ p (Sum.inl i)), ν (fun j ⇒ p (Sum.inr j))), ?_⟩
    funext x h
    rw [hS, hT]
    exact (ca_pair S μ T ν x h).symm⟩

/-- The product cone: `(A × B)^G` with its projections is a product of `A^G` and `B^G`. -/
def prodIsLimit (X Y : CAObj G) : IsLimit (BinaryFan.mk (fstHom X Y) (sndHom X Y)) :=
  BinaryFan.IsLimit.mk _ (fun f g ⇒ pairHom f g)
    (fun f g ⇒ by ext x h; simp [pairHom])
    (fun f g ⇒ by ext x h; simp [pairHom])
    (fun f g m h₁ h₂ ⇒ by
      ext x h
      have e₁ := congrFun (congrFun (congrArg Subtype.val h₁) x) h
      have e₂ := congrFun (congrFun (congrArg Subtype.val h₂) x) h
      simp only [CAObj.comp_val, Function.comp_apply, BinaryFan.mk_fst, BinaryFan.mk_snd,
        fstHom_apply, sndHom_apply] at e₁ e₂
      show m.1 x h = (f.1 x h, g.1 x h)
      exact Prod.ext e₁ e₂)

/-- The one-letter alphabet is terminal: the only map into it is a cellular automaton with an
empty memory set. -/
def unitIsTerminal : IsTerminal (unitObj : CAObj G) :=
  IsTerminal.ofUniqueHom
    (fun X ⇒ ⟨fun _ _ ⇒ PUnit.unit, ⟨PEmpty, inferInstance, PEmpty.elim,
      fun _ ⇒ PUnit.unit, rfl⟩⟩)
```

```
    (fun X m ⇒ by ext x h; rfl)

/-- **The cartesian category CA(G).** The cellular automata over `G` form a cartesian monoidal
category, with the product of alphabets as the monoidal product and the one-letter alphabet as
its unit: the structure the package's Catlab instance of `ThCartesianCategory` asserts. -/
instance : CartesianMonoidalCategory (CAObj G) :=
  .ofChosenFiniteProducts ⟨_, unitIsTerminal⟩ (fun X Y ⇒ ⟨_, prodIsLimit X Y⟩)

end CategoricalCellularAutomataProofs
```

Curtis–Hedlund–Lyndon

```
import CategoricalCellularAutomataProofs.Category
import Mathlib.Data.Finset.Basic
import Mathlib.Data.Fintype.Basic
```

The package decides whether a map of configurations is a cellular automaton (`is_equivariant`, `cellular_automaton`, `minimal_neighbourhood`) by checking that it commutes with the shifts and that its value at the identity depends on finitely many cells. This file proves that test correct, in the uniform form that needs no topology: over any group, a map of configurations (into a nonempty alphabet) is a cellular automaton exactly when it commutes with the shifts and its value at the identity depends only on the cells of some finite set. Over a finite group the second condition always holds, so a map is a cellular automaton exactly when it commutes with the shifts, with the whole group as a memory set.

```
namespace CategoricalCellularAutomataProofs

universe u

variable {G : Type u} [Monoid G] {A B : Type u}

/-- A map whose value at the identity depends only on the cells of `S`. -/
def LocalAt (S : Finset G) (f : Config G A → Config G B) : Prop :=
  ∀ x y : Config G A, (∀ s ∈ S, x s = y s) → f x 1 = f y 1

/-- A cellular automaton's value at the identity depends only on its memory set. -/
theorem IsCA.exists_localAt [DecidableEq G] {f : Config G A → Config G B} (hf : IsCA G f) :
    ∃ S : Finset G, LocalAt S f := by
  obtain ⟨I, _, S, μ, rfl⟩ := hf
  have := Fintype.ofFinite I
  refine ⟨Finset.univ.image S, fun x y h ⇒ ?_⟩
  simp only [ca_apply, mul_one]
  congr 1
  funext i
  exact h (S i) (Finset.mem_image_of_mem S (Finset.mem_univ i))

/-- **Curtis–Hedlund–Lyndon (uniform form).** Over any group, a map of configurations into
a nonempty alphabet is a cellular automaton exactly when it commutes with the shifts and its
value at the identity depends only on a finite set of cells; that set is then a memory set. -/
theorem isCA_iff_shift_comm_and_local [DecidableEq G] [Nonempty A]
    (f : Config G A → Config G B) :
    IsCA G f ↔ (∀ g x, f (shift g x) = shift g (f x)) ∧ ∃ S : Finset G, LocalAt S f := by
  constructor
  · intro hf
    exact ⟨hf.shift_comm, hf.exists_localAt⟩
  · rintro ⟨hshift, S, hS⟩
    classical
    let ext : (S → A) → Config G A := fun p g ⇒
      if h : g ∈ S then p ⟨g, h⟩ else Classical.arbitrary A
    refine ⟨S, inferInstance, fun s ⇒ s.1, fun p ⇒ f (ext p) 1, ?_⟩
    funext x g
    rw [apply_eq_apply_one_shift f hshift x g, ca_apply]
    refine hS _ _ (fun s hs ⇒ ?_)
    simp [ext, hs, shift]
```

```
/-- **Curtis–Hedlund–Lyndon on a finite group.** A map of configurations over a finite group
is a cellular automaton exactly when it commutes with the shifts; the whole group serves as
its memory set (`cellular_automaton` recovers the rule, `minimal_neighbourhood` shrinks the
set). -/
theorem isCA_iff_shift_comm [Finite G] (f : Config G A → Config G B) :
    IsCA G f ↔ ∀ g x, f (shift g x) = shift g (f x) := by
  constructor
  · exact fun hf ⇒ hf.shift_comm
  · intro hshift
    refine ⟨G, inferInstance, id, fun p ⇒ f p 1, ?_⟩
    funext x g
    rw [apply_eq_apply_one_shift f hshift x g]
    rfl

end CategoricalCellularAutomataProofs
```

Linear cellular automata and the group algebra

```
import CategoricalCellularAutomataProofs.Category
import Mathlib.Algebra.MonoidAlgebra.Defs
import Mathlib.Tactic.Ring
```

A linear cellular automaton with coefficients `a_s` maps `x` to `τ(x)(g) = Σ_s a_s x(s g)` (`LinearCA(G, coefficients; p)`, over `GF(p)`). The package composes linear automata and takes their powers through their coefficients, as elements of the group algebra, instead of composing local rules: `compose(L, M)` (first `L`, then `M`) has the coefficients of the product `L * M`. This file proves that this is right, over any commutative semiring and any group: running `a` and then `b` is the linear automaton of `a * b`, the unit of the algebra is the identity, and a linear automaton is a cellular automaton whose memory set is the support of its coefficients.

```
namespace CategoricalCellularAutomataProofs

universe u

variable {G : Type u} [Monoid G] {R : Type u} [CommSemiring R]

/-- The linear cellular automaton with coefficients `a`: `τ(x)(g) = Σ_s a_s x(s g)`. -/
noncomputable def linCA (a : G →₀ R) (x : Config G R) : Config G R :=
  fun g ⇒ a.sum fun s r ⇒ r * x (s * g)

theorem linCA_apply (a : G →₀ R) (x : Config G R) (g : G) :
    linCA a x g = a.sum fun s r ⇒ r * x (s * g) := rfl

/-- The convolution of coefficients, `(a ⋆ b)_u = Σ_{s t = u} a_s b_t`: the product of the
group algebra (`MonoidAlgebra.mul_def`). -/
noncomputable def conv (a b : G →₀ R) : G →₀ R :=
  a.sum fun s r ⇒ b.sum fun t r' ⇒ Finsupp.single (s * t) (r * r')

/-- Running `a` and then `b` is the linear automaton of their convolution. -/
theorem linCA_conv (a b : G →₀ R) (x : Config G R) :
    linCA b (linCA a x) = linCA (conv a b) x := by
  funext g
  simp only [linCA_apply, conv]
  rw [Finsupp.sum_sum_index (fun _ ⇒ zero_mul _) (fun _ _ _ ⇒ add_mul _ _ _)]
  have h1 : ∀ s r, ((b.sum fun t r' ⇒ Finsupp.single (s * t) (r * r')).sum
      fun u r ⇒ r * x (u * g)) = b.sum fun t r' ⇒ r * r' * x (s * t * g) := by
    intro s r
    rw [Finsupp.sum_sum_index (fun _ ⇒ zero_mul _) (fun _ _ _ ⇒ add_mul _ _ _)]
    exact Finsupp.sum_congr fun t _ ⇒ Finsupp.sum_single_index (zero_mul _)
  simp only [h1, Finsupp.mul_sum]
  rw [Finsupp.sum_comm]
  refine Finsupp.sum_congr fun s _ ⇒ Finsupp.sum_congr fun t _ ⇒ ?_
  rw [mul_assoc s t g]; ring
```

```
/-- The product of the group algebra is the convolution of coefficients. -/
theorem monoidAlgebra_mul_eq_conv (a b : MonoidAlgebra R G) :
    (a * b : MonoidAlgebra R G) = (conv a b : G →₀ R) := by
  rw [MonoidAlgebra.mul_def]; rfl

/-- **Composition through the coefficients.** Running `a` and then `b` is the linear automaton
of the product `a * b` in the group algebra (`compose(L, M)`). -/
theorem linCA_mul (a b : MonoidAlgebra R G) (x : Config G R) :
    linCA b (linCA a x) = linCA (a * b : MonoidAlgebra R G) x := by
  rw [linCA_conv, monoidAlgebra_mul_eq_conv]

/-- The unit of the group algebra is the identity automaton. -/
theorem linCA_one (x : Config G R) : linCA (1 : MonoidAlgebra R G) x = x := by
  funext g
  rw [MonoidAlgebra.one_def, linCA_apply, MonoidAlgebra.single, Finsupp.sum_single_index] <;>
    simp

/-- Powers through the coefficients: running `a` `n` times is the linear automaton of `a ^ n`
(`L^n`, computed by repeated squaring of the coefficients). -/
theorem linCA_pow (a : MonoidAlgebra R G) (n : ℕ) (x : Config G R) :
    (linCA a)^[n] x = linCA (a ^ n : MonoidAlgebra R G) x := by
  induction n generalizing x with
  | zero ⇒ simp [linCA_one]
  | succ n ih ⇒
    rw [Function.iterate_succ_apply', ih, linCA_mul, ← pow_succ]

/-- A linear automaton is a cellular automaton (`CellularAutomaton(L)`): its memory set is the
support of its coefficients and its local rule the weighted sum. -/
theorem isCA_linCA (a : G →₀ R) : IsCA G (linCA a) := by
  classical
  refine ⟨a.support, inferInstance, fun s ⇒ s.1,
    fun p ⇒ ∑ s : a.support, a s.1 * p s, ?_⟩
  funext x g
  simp only [linCA_apply, ca_apply, Finsupp.sum]
  exact (Finset.sum_coe_sort a.support (fun s ⇒ a s * x (s * g))).symm

end CategoricalCellularAutomataProofs
```

Generalised cellular automata and pullbacks

```
import CategoricalCellularAutomataProofs.CellularAutomata
```

For a homomorphism `φ : H →* G`, a generalised cellular automaton maps configurations over `G` to configurations over `H` by `T(x)(h) = μ((x(s φ(h)))_{s ∈ S})` (`GeneralizedCA(φ, S, μ)`).

This file proves the facts the Julia package relies on:

- `T` is the pullback along `φ` after the ordinary cellular automaton with the same memory set and rule (`factorize`);
- **pullback commutes with local rules**: pulling back the output of a cellular automaton over `G` whose memory set is the image `φ(S)` is the same as running the cellular automaton over `H` with memory set `S` on the pulled-back configuration;
- `T` intertwines the shifts along `φ`;
- two generalised automata compose to one, along the composite homomorphism, with memory set the products `S i * φ(T j)` (the set `T_φ(S)` of `compose`).

```
namespace CategoricalCellularAutomataProofs

variable {G H K A B C : Type*} {I J : Type*}

section
```

```
variable [Monoid G] [Monoid H] [Monoid K]

/-- The generalised cellular automaton along `φ` with memory set `S` and local rule `μ`. -/
def gca (φ : H →* G) (S : I → G) (μ : (I → A) → B) (x : Config G A) : Config H B :=
  fun h ⇒ μ (fun i ⇒ x (S i * φ h))

@[simp] theorem gca_apply (φ : H →* G) (S : I → G) (μ : (I → A) → B) (x : Config G A)
    (h : H) : gca φ S μ x h = μ (fun i ⇒ x (S i * φ h)) := rfl

/-- **Factorisation.** `T = φ* ∘ τ` (`factorize(T) == (φ, τ)`). -/
theorem gca_eq_pullback_ca (φ : H →* G) (S : I → G) (μ : (I → A) → B) (x : Config G A) :
    gca φ S μ x = pullback φ (ca S μ x) := rfl

/-- An ordinary cellular automaton is the generalised one over the identity homomorphism
(`GeneralizedCA(τ)`). -/
theorem gca_id (S : I → G) (μ : (I → A) → B) (x : Config G A) :
    gca (MonoidHom.id G) S μ x = ca S μ x := rfl

/-- **Pullback commutes with local rules.** For a memory set `S` in `H`, running the cellular
automaton over `G` with memory set `φ(S)` and pulling back equals pulling back and running the
cellular automaton over `H` with memory set `S`: `φ* ∘ τ_{φ(S)} = τ_S ∘ φ*`. -/
theorem pullback_ca (φ : H →* G) (S : I → H) (μ : (I → A) → B) (x : Config G A) :
    pullback φ (ca (fun i ⇒ φ (S i)) μ x) = ca S μ (pullback φ x) := by
  funext h; simp [pullback, ca, map_mul]

/-- The same, as an equality of generalised and ordinary automata:
`T_{φ, φ(S)} = τ_S ∘ φ*`. -/
theorem gca_map_eq_ca_pullback (φ : H →* G) (S : I → H) (μ : (I → A) → B)
    (x : Config G A) :
    gca φ (fun i ⇒ φ (S i)) μ x = ca S μ (pullback φ x) :=
  pullback_ca φ S μ x

/-- A generalised automaton intertwines the shifts along `φ`: `T (φ(h) · x) = h · T x`. -/
theorem gca_shift (φ : H →* G) (S : I → G) (μ : (I → A) → B) (h : H) (x : Config G A) :
    gca φ S μ (shift (φ h) x) = shift h (gca φ S μ x) := by
  funext k; simp [gca, shift, map_mul, mul_assoc]

/-- **Composition.** For `φ : H →* G` and `ψ : K →* H`, running `T = gca φ S μ` and then
`U = gca ψ T ν` is the generalised automaton along `φ ∘ ψ` whose memory set is the family of
products `S i * φ(T j)`, with the local rule applying `μ` to each block and `ν` to the
results. -/
theorem gca_comp (φ : H →* G) (S : I → G) (μ : (I → A) → B) (ψ : K →* H) (T : J → H)
    (ν : (J → B) → C) (x : Config G A) :
    gca ψ T ν (gca φ S μ x) =
      gca (φ.comp ψ) (fun p : J × I ⇒ S p.2 * φ (T p.1))
        (fun q ⇒ ν (fun j ⇒ μ (fun i ⇒ q (j, i)))) x := by
  funext k; simp [gca, map_mul, mul_assoc]

end

end CategoricalCellularAutomataProofs
```

Gluing spaces

```
import Mathlib.CategoryTheory.Limits.IsLimit
import Mathlib.CategoryTheory.Types.Basic
import Mathlib.Data.List.Basic
```

`glue` builds a landscape as the colimit of patches: each patch is a set of cells (with the slots its interior cells read), the patches share cells along their feet, and the landscape's cells are the colimit of the patches' cells. This file proves the two facts the Julia tests check numerically.

Configurations form a limit. A configuration of the landscape is a function from its cells to the alphabet, a morphism out of the colimit in `Type`. So configurations of the landscape are exactly the families of configurations

of the patches that agree wherever patches share cells: the colimit of cells becomes a limit of configurations. Any colimit of cell sets will do: Catlab's colimit along an undirected wiring diagram and the package's union-find give the same one up to isomorphism.

A step restricted to a patch is the patch's step. A cell reads the cells of its slots, in slot order (the `index` attribute), and a local rule maps those values to the cell's next value. A patch's interior cells keep all their slots: the map of the patch into the landscape sends the reads of each interior cell, in order, to the reads of its image. Then one step of the landscape, read on a patch's interior, is the patch's own step on the restricted configuration (halo exchange), and the steps of patches whose interiors cover the landscape determine the landscape's step.

```
namespace CategoricalCellularAutomataProofs

open CategoryTheory Limits

universe u

section Limit

variable {J : Type u} [SmallCategory J] {F : J ⥤ Type u} {c : Cocone F}

/-- Configurations of the glued space (functions from its cells to `A`) are the compatible
families of configurations of the patches: a cocone over the patches' cells with apex `A`. -/
def gluedConfigEquiv (hc : IsColimit c) (A : Type u) :
    (c.pt → A) ≃ (F ⟶ (Functor.const J).obj A) :=
  hc.homEquiv

/-- The family a configuration of the glued space gives is its restriction to each patch. -/
theorem gluedConfigEquiv_app (hc : IsColimit c) (A : Type u) (x : c.pt → A) (j : J) :
    (gluedConfigEquiv hc A x).app j = c.ι.app j ≫ x :=
  rfl

/-- **Unique gluing.** Configurations of the patches that agree along every map of the
diagram (on shared cells) are the restrictions of exactly one configuration of the glued
space. -/
theorem exists_unique_glued_config (hc : IsColimit c) {A : Type u} (xs : ∀ j, F.obj j → A)
    (compat : ∀ {j k : J} (f : j ⟶ k), F.map f ≫ xs k = xs j) :
    ∃! x : c.pt → A, ∀ j, c.ι.app j ≫ x = xs j := by
  let s : Cocone F :=
    { pt := A
      ι := { app := xs
             naturality := fun j k f ⇒ by simpa using compat f } }
  exact hc.existsUnique s

/-- Two configurations of the glued space that agree on every patch are equal. -/
theorem glued_config_ext (hc : IsColimit c) {A : Type u} {x y : c.pt → A}
    (h : ∀ j, c.ι.app j ≫ x = c.ι.app j ≫ y) : x = y :=
  hc.hom_ext h

end Limit

section Step

variable {X P Q A : Type*}

/-- One synchronous step on a space whose cell `c` reads the cells `nbrs c` in slot order:
the local rule `μ c` maps their values to the next value of `c`. -/
def step (nbrs : X → List X) (μ : X → List A → A) (x : X → A) : X → A :=
  fun c ⇒ μ c ((nbrs c).map x)

/-- A patch of the landscape: the patch's cells `P` with a map `f` into the landscape's
cells, and its interior cells `Q`, included by `ι`, each reading `nbrsP` in slot order. The
map is a map of spaces on the interior: it sends the reads of an interior cell, in order, to
the reads of its image. -/
structure Patch (X P Q : Type*) (nbrs : X → List X) where
  /-- the patch's cells in the landscape -/
  f : P → X
```

```
  /-- the interior cells among the patch's cells -/
  ι : Q → P
  /-- the reads of each interior cell, in slot order -/
  nbrsP : Q → List P
  /-- reads are preserved, in order -/
  map_nbrs : ∀ q, (nbrsP q).map f = nbrs (f (ι q))

/-- The patch's own step on its interior, from a configuration of the patch's cells (interior
and halo); the rule of an interior cell is the rule of its image. -/
def Patch.step {nbrs : X → List X} (p : Patch X P Q nbrs) (μ : X → List A → A) (z : P → A) :
    Q → A :=
  fun q ⇛ μ (p.f (p.ι q)) ((p.nbrsP q).map z)

/-- **Halo exchange.** One step of the landscape, read on a patch's interior, is the patch's
own step on the configuration restricted to the patch. -/
theorem step_restrict {nbrs : X → List X} (p : Patch X P Q nbrs) (μ : X → List A → A)
    (x : X → A) (q : Q) :
    step nbrs μ x (p.f (p.ι q)) = p.step μ (x ∘ p.f) q := by
  simp only [step, Patch.step, ← p.map_nbrs, List.map_map]

/-- **Locality.** The landscape's next values on a patch's interior depend only on the
configuration on the patch. -/
theorem step_local {nbrs : X → List X} (p : Patch X P Q nbrs) (μ : X → List A → A)
    {x y : X → A} (h : x ∘ p.f = y ∘ p.f) (q : Q) :
    step nbrs μ x (p.f (p.ι q)) = step nbrs μ y (p.f (p.ι q)) := by
  rw [step_restrict, step_restrict, h]

/-- **The patches determine the step.** If the interiors of a family of patches cover the
landscape, the landscape's step is the unique configuration whose value on each patch's
interior is that patch's step on the restriction. -/
theorem step_eq_of_cover {ι' : Type*} {P Q : ι' → Type*} {nbrs : X → List X}
    (ps : ∀ k, Patch X (P k) (Q k) nbrs) (μ : X → List A → A)
    (cover : ∀ c, ∃ k q, (ps k).f ((ps k).ι q) = c) (x y : X → A)
    (hy : ∀ k q, y ((ps k).f ((ps k).ι q)) = (ps k).step μ (x ∘ (ps k).f) q) :
    y = step nbrs μ x := by
  funext c
  obtain ⟨k, q, rfl⟩ := cover c
  rw [hy, step_restrict]

end Step

end CategoricalCellularAutomataProofs
```

Moving models between spaces

```
import CategoricalCellularAutomataProofs.Gluing
```

A space is a set of cells `X`, each reading its neighbours `nbrs c` in slot order (a list, so graphs, whose cells have different numbers of neighbours, are included). A map of spaces `φ : Y → X` preserves neighbours up to a reordering of each cell's slots: `ρ y` puts the images of `y`'s neighbours in the slot order of `φ y`. A reordering moves list entries by position, so it commutes with relabelling them (`ρ_natural`): a natural transformation of the list functor, such as the identity (a covering) or reversal (a one-dimensional fold).

The package's `pullback(f, prob; space)` moves a problem along such a map: each cell `y` of the new space runs the rule of its image `φ y`, reading its neighbours reordered by `ρ y`, and draws the random numbers of `φ y`. A rule that depends on the cell and the time (as a rule drawing counter-based random numbers keyed by cell and step does) is a family `μ t c : List A → A`. The pulled-back run is the pullback of the run, step by step, for every such rule: no symmetry of the rule is needed.

```
namespace CategoricalCellularAutomataProofs

universe u
```

```
variable {X Y A : Type u}

/-- The run of a time-dependent rule: `run nbrs μ x n` is the configuration after `n`
synchronous steps from `x`, step `t` applying the rule `μ t`. -/
def run (nbrs : X → List X) (μ : ℕ → X → List A → A) (x : X → A) : ℕ → X → A
  | 0 ⇒ x
  | n + 1 ⇒ step nbrs (μ n) (run nbrs μ x n)

/-- A map of spaces `φ : Y → X` with slot reorderings `ρ`. -/
structure SpaceMap (Y X : Type u) (nbrsY : Y → List Y) (nbrsX : X → List X) where
  /-- where each cell goes -/
  φ : Y → X
  /-- the reordering of each cell's slots, at every type of values -/
  ρ : Y → ∀ {B : Type u}, List B → List B
  /-- a reordering moves entries by position, so it commutes with relabelling them -/
  ρ_natural : ∀ (y : Y) {B C : Type u} (g : B → C) (l : List B), ρ y (l.map g) = (ρ y l).map g
  /-- neighbours are preserved: the images of `y`'s neighbours, reordered, are `φ y`'s -/
  map_nbrs : ∀ y, ρ y ((nbrsY y).map φ) = nbrsX (φ y)

variable {nbrsY : Y → List Y} {nbrsX : X → List X}

/-- The rule pulled back along a map of spaces: the cell `y` runs the rule of `φ y` on its
neighbours' values in `φ y`'s slot order. -/
def SpaceMap.pullRule (f : SpaceMap Y X nbrsY nbrsX) (μ : X → List A → A) : Y → List A → A :=
  fun y l ⇒ μ (f.φ y) (f.ρ y l)

/-- One step of the pulled-back rule from a pulled-back configuration is the pullback of one
step. -/
theorem SpaceMap.step_pullback (f : SpaceMap Y X nbrsY nbrsX) (μ : X → List A → A)
    (x : X → A) : step nbrsY (f.pullRule μ) (x ∘ f.φ) = step nbrsX μ x ∘ f.φ := by
  funext y
  simp only [step, SpaceMap.pullRule, Function.comp_apply]
  rw [← List.map_map, f.ρ_natural, f.map_nbrs]

/-- **Moving a model along a map of spaces.** The run of the pulled-back time-dependent rule
from the pulled-back initial state is, at every step, the pullback of the run. -/
theorem SpaceMap.run_pullback (f : SpaceMap Y X nbrsY nbrsX) (μ : ℕ → X → List A → A)
    (x : X → A) (n : ℕ) :
    run nbrsY (fun t ⇒ f.pullRule (μ t)) (x ∘ f.φ) n = run nbrsX μ x n ∘ f.φ := by
  induction n with
  | zero ⇒ rfl
  | succ n ih ⇒ simp only [run, ih, SpaceMap.step_pullback]

/-- A covering: a map that preserves neighbours slot by slot (no reordering). -/
def SpaceMap.ofSlotwise (φ : Y → X) (h : ∀ y, (nbrsY y).map φ = nbrsX (φ y)) :
    SpaceMap Y X nbrsY nbrsX where
  φ := φ
  ρ _ _ l := l
  ρ_natural _ _ _ _ _ := rfl
  map_nbrs := h

/-- A fold that reverses the slots of some cells (on a line with neighbours `[left, right]`,
the mirrored half reads them the other way round), as `reflection_fold` builds. -/
def SpaceMap.ofReversing (φ : Y → X) (flip : Y → Bool)
    (h : ∀ y, (if flip y then List.reverse else id) ((nbrsY y).map φ) = nbrsX (φ y)) :
    SpaceMap Y X nbrsY nbrsX where
  φ := φ
  ρ y _ l := if flip y then l.reverse else l
  ρ_natural y _ _ g l := by cases flip y <;> simp [List.map_reverse]
  map_nbrs y := by simpa [apply_ite (fun F : List X → List X ⇒ F ((nbrsY y).map φ))] using h y

end CategoricalCellularAutomataProofs
```